SHERLOCK

Security Review For Kaskad



Collaborative Audit Prepared For: **Kaskad**
Lead Security Expert(s): **AV**
KupiaSec
Date Audited: **September 23 - September 25, 2026**

Introduction

Pontifex is Kaskad's one-way KSKD migration bridge from Igra to Robinhood Chain.

This Sherlock audit covers the Pontifex onchain contracts in the Kaskad-Lending/kaskad-pontifex-contracts repository, at commit b0780f33f761b931f82dc4c040b611bc7c876a08.

The scoped contracts implement the following bridge-side functionality:

- Burning KSKD on Igra through KskdExit.
- Minting bridged KSKD on Robinhood through KskdEntry and KskdBridged.
- Verifying cumulative recipient claims so repeated submissions cannot mint more than the difference between the recipient's burned amount and what has already been bridged in.
- Enforcing mint controls such as admitted signer checks and daily mint limits.

The reviewed scope consists of 4 Solidity files totaling 226 nSLOC: `KskdExit.sol`, `KskdEntry.sol`, `KskdBridged.sol`, and `IAttestationVerifier.sol`

Scope

Repository: Kaskad-Lending/kaskad-pontifex-contracts

Audited Commit: b0780f33f761b931f82dc4c040b611bc7c876a08

Final Commit: b0780f33f761b931f82dc4c040b611bc7c876a08

Files:

- src/interfaces/IAttestationVerifier.sol
- src/KskdBridged.sol
- src/KskdEntry.sol
- src/KskdExit.sol

Final Commit Hash

b0780f33f761b931f82dc4c040b611bc7c876a08

Findings

Each issue has an assigned severity:

- High issues are directly exploitable security vulnerabilities that need to be fixed.

- Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.
- Low/Info issues are non-exploitable, informational findings that do not pose a security risk or impact the system's integrity. These issues are typically cosmetic or related to compliance requirements, and are not considered a priority for remediation.

Issues Found

High	Medium	Low/Info
0	0	3

Issues Not Fixed and Not Acknowledged

High	Medium	Low/Info
0	0	0

Issue L-1: `setBridge` does not verify the replacement entry's token [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit2/2026-09-kaskad-sept-23rd-2026/issues/5

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

`setBridge()` does not check that a replacement entry's `TOKEN()` points to the existing token.

Vulnerability Detail

If an operator mistakenly deploys a new token and entry pair, then points the original token at that entry, claims through the new entry mint its new token once the claim prerequisites are met. The former entry loses authority to mint the original token.

Impact

The owner can restore the original bridge pointer while retaining ownership. Tokens already minted by the mistaken new pair remain issued; the scoped contracts provide no clawback.

Code Snippet

From KskdBridged.sol:

```
function setBridge(address newBridge) external onlyOwner {
    bridge = newBridge;
    emit BridgeSet(newBridge);
}
```

Tool Used

Manual Review

Recommendation

For nonzero replacements, check that the entry's `TOKEN()` equals `address(this)`, while preserving `setBridge(0)`. If arbitrary bridge implementations must remain supported, make this check in the replacement script or Safe transaction bundle.

Issue L-2: `NothingToMint` compares against minted amount, not prior signatures [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit2/2026-09-kaskad-sept-23rd-2026/issues/6

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

Documentation says re-signing a lower or equal cumulative causes `NothingToMint`. The contract compares that cumulative with the amount **already minted**, rather than an earlier signed value.

Vulnerability Detail

After a signed cumulative of 100 partially mints 40, the same cumulative can later mint up to the remaining 60 with valid signatures and available capacity.

Impact

The wording may mislead an operator or integrator about claiming an unpaid remainder.

Code Snippet

From KskdEntry.sol:

```
uint256 prior = TOKEN.bridgedIn(recipient);
if (cumulativeBurned <= prior) revert KskdBridged.NothingToMint(cumulativeBurned,
↪  prior);
uint256 minted = _reserve(cumulativeBurned - prior);
```

Tool Used

Manual Review

Recommendation

State that `NothingToMint` occurs when the signed cumulative is no greater than `bridgedIn[recipient]`. Clarify that a partial remainder can be claimed with valid signatures and capacity, and that an expired signature needs renewal.

Issue L-3: Redundant `NothingToMint` check in `KskdBridged.bridgeMint()` function [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit2/2026-09-kaskad-sept-23rd-2026/issues/7

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

The `cumulative <= prior` check in `KskdBridged.bridgeMint()` is unreachable because the bridge always passes `cumulative = prior + minted` with `minted > 0`, making the revert at L51 redundant.

Vulnerability Detail

The `KskdBridged.bridgeMint()` function reverts when `cumulative` is less than or equal to `prior`:

```
47:     function bridgeMint(address recipient, uint256 cumulative) external {
48:         if (msg.sender != bridge) revert NotBridge();
49:
50:         uint256 prior = bridgedIn[recipient];
51: @>      if (cumulative <= prior) revert NothingToMint(cumulative, prior);
```

However, this condition can never be true. `bridgeMint()` is only callable by `bridge`, and the bridge computes `cumulative` as `prior + minted`, where `minted` is the return value of `_reserve()` and is always greater than zero. As a result, `cumulative` is always strictly greater than `prior` by the time `bridgeMint()` is invoked, and the check at L51 is unreachable:

```
184:         uint256 prior = TOKEN.bridgedIn(recipient);
185:         if (cumulativeBurned <= prior) revert
 ↪  KskdBridged.NothingToMint(cumulativeBurned, prior);
186:         uint256 minted = _reserve(cumulativeBurned - prior);
187:         uint256 cumulative = prior + minted;
188:
189:         TOKEN.bridgeMint(recipient, cumulative);
190:         emit Claimed(recipient, minted, cumulative, cumulativeBurned);
```

Impact

Code improvement – the check is redundant and adds unnecessary gas and complexity.

Code Snippet

https://github.com/sherlock-audit2/2026-09-kaskad-sept-23rd-2026/blob/f9631717b47085db4684ae8ce4ae62cf5332ca26/kaskad-pontifex-contracts/src/KskdBridged.sol#L51

Tool Used

Manual Review

Recommendation

Remove the redundant revert at L51 of `bridgeMint()`, since the equivalent check is already enforced by the bridge at L185.

Disclaimers

Sherlock does not provide guarantees nor warranties relating to the security of the project.

Usage of all smart contract software is at the respective users' sole risk and is the users' responsibility.